

Geometry of Deep Learning

Patrícia Muñoz Ewald

University of Texas at Austin

November 11, 2025

About me: Physics (BS) $\longrightarrow$ Mathematics (MS, “*Compactness in gauge theory*”) $\longrightarrow$ PhD candidate in Mathematics.
TQFT $\longrightarrow$ OT $\longrightarrow$ RF+RG $\longrightarrow$ DL

Main interest: Mathematical foundations of deep learning.

1 Truncation map

- Theory
- Application: Symmetry
- Application: Classification
- Experiments

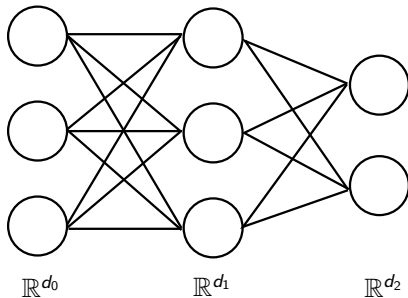
2 Adapted gradient flow

Includes joint work with Thomas Chen and Mayank Konduri (UT Austin).

Consider a neural network

$$\begin{aligned}x^{(0)} &= x_0 \in \mathbb{R}^{d_0} \text{ initial input,} \\x^{(\ell)} &= \sigma(W_\ell x^{(\ell-1)} + b_\ell) \in \mathbb{R}^{d_\ell}, \text{ for } \ell = 1, \dots, L-1, \\x^{(L)} &= W_L x^{(L-1)} + b_L \in \mathbb{R}^{d_L}.\end{aligned}\tag{3.1}$$

Usually, will take $\sigma(x)_i = \max\{0, x_i\}$ (ReLU).

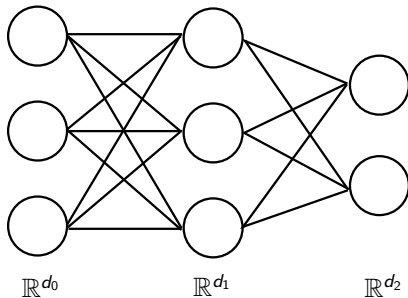


Goal: Directly compare features or representations of data in different layers.

Consider a neural network

$$\begin{aligned} x^{(0)} &= x_0 \in \mathbb{R}^{d_0} \text{ initial input,} \\ x^{(\ell)} &= \sigma(W_\ell x^{(\ell-1)} + b_\ell) \in \mathbb{R}^{d_\ell}, \text{ for } \ell = 1, \dots, L-1, \\ x^{(L)} &= W_L x^{(L-1)} + b_L \in \mathbb{R}^{d_L}. \end{aligned} \quad (3.1)$$

Usually, will take $\sigma(x)_i = \max\{0, x_i\}$ (ReLU).



Goal: Directly compare features or representations of data in different layers.

Consider a neural network

$$\begin{aligned}x^{(0)} &= x_0 \in \mathbb{R}^{d_0} \text{ initial input,} \\x^{(\ell)} &= \sigma(W_\ell x^{(\ell-1)} + b_\ell) \in \mathbb{R}^{d_\ell}, \text{ for } \ell = 1, \dots, L-1, \\x^{(L)} &= W_L x^{(L-1)} + b_L \in \mathbb{R}^{d_L}.\end{aligned}\tag{3.2}$$

Usually, will take $\sigma(x)_i = \max\{0, x_i\}$ (ReLU).



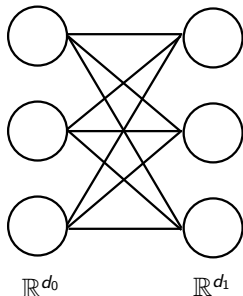
$\mathbb{R}^{d_0}$

Goal: Directly compare features or representations of data in different layers.

Consider a neural network

$$\begin{aligned} x^{(0)} &= x_0 \in \mathbb{R}^{d_0} \quad \text{initial input,} \\ x^{(\ell)} &= \sigma(W_\ell x^{(\ell-1)} + b_\ell) \in \mathbb{R}^{d_\ell}, \quad \text{for } \ell = 1, \dots, L-1, \\ x^{(L)} &= W_L x^{(L-1)} + b_L \in \mathbb{R}^{d_L}. \end{aligned} \quad (3.3)$$

Usually, will take $\sigma(x)_i = \max\{0, x_i\}$ (ReLU).

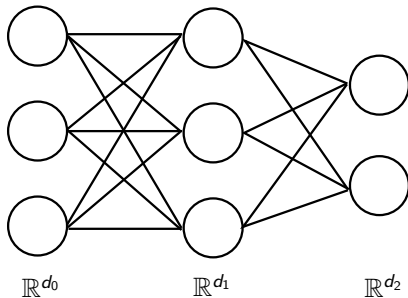


Goal: Directly compare features or representations of data in different layers.

Consider a neural network

$$\begin{aligned} x^{(0)} &= x_0 \in \mathbb{R}^{d_0} \text{ initial input,} \\ x^{(\ell)} &= \sigma(W_\ell x^{(\ell-1)} + b_\ell) \in \mathbb{R}^{d_\ell}, \text{ for } \ell = 1, \dots, L-1, \\ x^{(L)} &= W_L x^{(L-1)} + b_L \in \mathbb{R}^{d_L}. \end{aligned} \quad (3.4)$$

Usually, will take $\sigma(x)_i = \max\{0, x_i\}$ (ReLU).

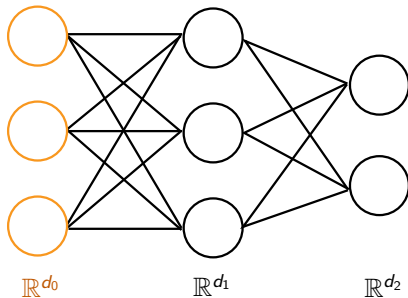


Goal: Directly compare features or representations of data in different layers.

Consider a neural network

$$\begin{aligned} x^{(0)} &= x_0 \in \mathbb{R}^{d_0} \text{ initial input,} \\ x^{(\ell)} &= \sigma(W_\ell x^{(\ell-1)} + b_\ell) \in \mathbb{R}^{d_\ell}, \text{ for } \ell = 1, \dots, L-1, \\ x^{(L)} &= W_L x^{(L-1)} + b_L \in \mathbb{R}^{d_L}. \end{aligned} \quad (3.5)$$

Usually, will take $\sigma(x)_i = \max\{0, x_i\}$ (ReLU).

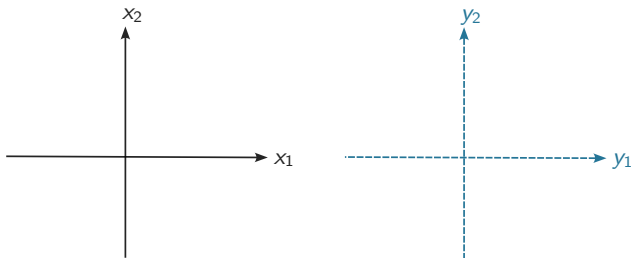


Goal: Directly compare features or representations of data in different layers.

Definition (Chen–E. '23)

Given $W \in \mathbb{R}^{d_1 \times d_0}$ and $b \in \mathbb{R}^{d_1}$ we define the **truncation map**

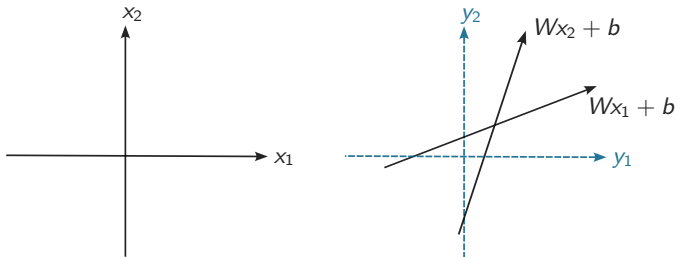
$$\begin{aligned}\tau_{W,b} : \mathbb{R}^{d_0} &\rightarrow \mathbb{R}^{d_0}, \\ x &\mapsto W^+ (\sigma(Wx + b) - b).\end{aligned}$$



Definition (Chen–E. '23)

Given $W \in \mathbb{R}^{d_1 \times d_0}$ and $b \in \mathbb{R}^{d_1}$ we define the **truncation map**

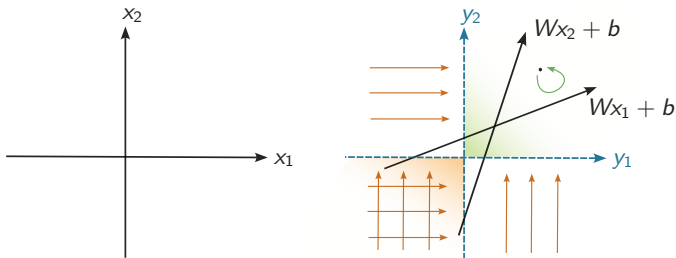
$$\begin{aligned}\tau_{W,b} : \mathbb{R}^{d_0} &\rightarrow \mathbb{R}^{d_0}, \\ x &\mapsto W^+ (\sigma(Wx + b) - b).\end{aligned}$$



Definition (Chen–E. '23)

Given $W \in \mathbb{R}^{d_1 \times d_0}$ and $b \in \mathbb{R}^{d_1}$ we define the **truncation map**

$$\begin{aligned}\tau_{W,b} : \mathbb{R}^{d_0} &\rightarrow \mathbb{R}^{d_0}, \\ x &\mapsto W^+ (\sigma(Wx + b) - b).\end{aligned}$$

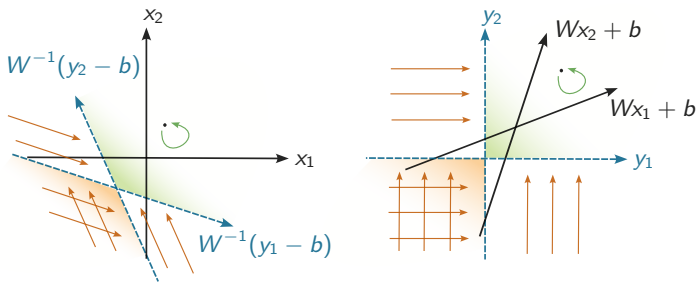


Definition (Chen–E. '23)

Given $W \in \mathbb{R}^{d_1 \times d_0}$ and $b \in \mathbb{R}^{d_1}$ we define the **truncation map**

$$\tau_{W,b} : \mathbb{R}^{d_0} \rightarrow \mathbb{R}^{d_0},$$

$$x \mapsto W^+ (\sigma(Wx + b) - b).$$



Define **cumulative parameters**

$$\begin{aligned} W^{(1)} &:= W_1, & W^{(\ell)} &:= W_\ell \cdots W_1 = W_\ell W^{(\ell-1)}, \\ b^{(1)} &:= b_1, & b^{(\ell)} &:= W_\ell b^{(\ell-1)} + b_\ell, \end{aligned} \quad (3.6)$$

and

$$x^{(\tau,0)} := x^{(0)}, \quad x^{(\tau,\ell)} := \tau_{W^{(\ell)}, b^{(\ell)}}(x^{(\tau,\ell-1)}). \quad (3.7)$$

Proposition (Chen–E. '24)

If all weight matrices W_ℓ are surjective, $\ell = 1, \dots, L$, then

$$x^{(\ell)} = W^{(\ell)} x^{(\tau,\ell)} + b^{(\ell)}, \quad (3.8)$$

for $\ell = 1, \dots, L-1$, and

$$x^{(L)} = W^{(L)} x^{(\tau,L-1)} + b^{(L)}. \quad (3.9)$$

Define **cumulative parameters**

$$\begin{aligned} W^{(1)} &:= W_1, & W^{(\ell)} &:= W_\ell \cdots W_1 = W_\ell W^{(\ell-1)}, \\ b^{(1)} &:= b_1, & b^{(\ell)} &:= W_\ell b^{(\ell-1)} + b_\ell, \end{aligned} \tag{3.6}$$

and

$$x^{(\tau,0)} := x^{(0)}, \quad x^{(\tau,\ell)} := \tau_{W^{(\ell)}, b^{(\ell)}}(x^{(\tau,\ell-1)}). \tag{3.7}$$

Proposition (Chen–E. '24)

If all weight matrices W_ℓ are surjective, $\ell = 1, \dots, L$, then

$$x^{(\ell)} = W^{(\ell)} x^{(\tau,\ell)} + b^{(\ell)}, \tag{3.8}$$

for $\ell = 1, \dots, L-1$, and

$$x^{(L)} = W^{(L)} x^{(\tau, L-1)} + b^{(L)}. \tag{3.9}$$

This means that a feedforward neural network

$$\underline{f}_\theta : \underbrace{\mathbb{R}^{d_0}}_{\text{Input layer}} \xrightarrow{f_1} \underbrace{\mathbb{R}^{d_1} \xrightarrow{f_2} \dots \xrightarrow{f_{L-1}} \mathbb{R}^{d_{L-1}}}_{\text{Hidden layers}} \xrightarrow{f_L} \underbrace{\mathbb{R}^{d_L}}_{\text{Output layer}}, \quad (3.10)$$

can be rewritten as a sequence of transformations on input space, followed by an affine map to output space,

$$\underline{f}_\theta : \underbrace{\mathbb{R}^{d_0} \xrightarrow{\tau_1} \mathbb{R}^{d_0} \xrightarrow{\tau_2} \dots \xrightarrow{\tau_{L-1}} \mathbb{R}^{d_0}}_{\text{Truncation maps}} \xrightarrow{\tilde{f}_L} \mathbb{R}^{d_L}. \quad (3.11)$$

This is independent of the activation function(s) being used.

(E. 2025) This result can be adapted for arbitrary feedforward networks.

This means that a feedforward neural network

$$\underline{f}_{\underline{\theta}} : \underbrace{\mathbb{R}^{d_0}}_{\text{Input layer}} \xrightarrow{f_1} \underbrace{\mathbb{R}^{d_1} \xrightarrow{f_2} \dots \xrightarrow{f_{L-1}} \mathbb{R}^{d_{L-1}}}_{\text{Hidden layers}} \xrightarrow{f_L} \underbrace{\mathbb{R}^{d_L}}_{\text{Output layer}}, \quad (3.10)$$

can be rewritten as a sequence of transformations on input space, followed by an affine map to output space,

$$\underline{f}_{\underline{\theta}} : \underbrace{\mathbb{R}^{d_0} \xrightarrow{\tau_1} \mathbb{R}^{d_0} \xrightarrow{\tau_2} \dots \xrightarrow{\tau_{L-1}} \mathbb{R}^{d_0}}_{\text{Truncation maps}} \xrightarrow{\tilde{f}_L} \mathbb{R}^{d_L}. \quad (3.11)$$

This is independent of the activation function(s) being used.

(E. 2025) This result can be adapted for arbitrary feedforward networks.

This means that a feedforward neural network

$$\underline{f}_{\underline{\theta}} : \underbrace{\mathbb{R}^{d_0}}_{\text{Input layer}} \xrightarrow{f_1} \underbrace{\mathbb{R}^{d_1} \xrightarrow{f_2} \dots \xrightarrow{f_{L-1}} \mathbb{R}^{d_{L-1}}}_{\text{Hidden layers}} \xrightarrow{f_L} \underbrace{\mathbb{R}^{d_L}}_{\text{Output layer}}, \quad (3.10)$$

can be rewritten as a sequence of transformations on input space, followed by an affine map to output space,

$$\underline{f}_{\underline{\theta}} : \underbrace{\mathbb{R}^{d_0} \xrightarrow{\tau_1} \mathbb{R}^{d_0} \xrightarrow{\tau_2} \dots \xrightarrow{\tau_{L-1}} \mathbb{R}^{d_0}}_{\text{Truncation maps}} \xrightarrow{\tilde{f}_L} \mathbb{R}^{d_L}. \quad (3.11)$$

This is independent of the activation function(s) being used.

(E. 2025) This result can be adapted for arbitrary feedforward networks.

Potential/work in progress.

Deep neural networks have very large symmetry groups. For instance, permutation of neurons within a layer. The set of internal symmetries is given by the intertwiner group:

$$G_{\sigma,n} := \left\{ A \in GL(n, \mathbb{R}) : \text{there exists a } B \in GL(n, \mathbb{R}) \quad (3.12) \right. \\ \left. \text{such that } \sigma \circ A = B \circ \sigma \right\}.$$

When $B = A$, the truncation map is invariant under the intertwiner group, at least when the weight matrices are invertible.

Results on representational alignment are data dependent.

Potential/work in progress.

Deep neural networks have very large symmetry groups. For instance, permutation of neurons within a layer. The set of internal symmetries is given by the intertwiner group:

$$G_{\sigma,n} := \left\{ A \in GL(n, \mathbb{R}) : \text{there exists a } B \in GL(n, \mathbb{R}) \quad (3.12) \right. \\ \left. \text{such that } \sigma \circ A = B \circ \sigma \right\}.$$

When $B = A$, the truncation map is invariant under the intertwiner group, at least when the weight matrices are invertible.

Results on representational alignment are data dependent.

Potential/work in progress.

Deep neural networks have very large symmetry groups. For instance, permutation of neurons within a layer. The set of internal symmetries is given by the intertwiner group:

$$G_{\sigma,n} := \left\{ A \in GL(n, \mathbb{R}) : \text{there exists a } B \in GL(n, \mathbb{R}) \quad (3.12) \right. \\ \left. \text{such that } \sigma \circ A = B \circ \sigma \right\}.$$

When $B = A$, the truncation map is invariant under the intertwiner group, at least when the weight matrices are invertible.

Results on representational alignment are data dependent.

Truncation map and (round) cones for ReLU

Truncation
map

Theory

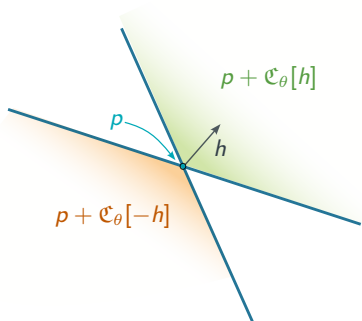
Application:
Symmetry

Application:
Classification

Experiments

Adapted
gradient flow

Consider the cone $p + \mathfrak{C}_\theta[h] \subseteq \mathbb{R}^n$ centered around a unit vector h and based at p :



There are W and b such that, for $\sigma = \text{ReLU}$,

$$\tau_{W,b}(x) = \begin{cases} x, & x \in p + \mathfrak{C}_\theta[h], \\ p, & x \in p + \mathfrak{C}_\theta[-h]. \end{cases} \quad (3.13)$$

Truncation map and (round) cones for ReLU

Truncation
map

Theory

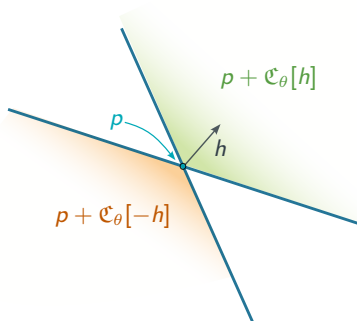
Application:
Symmetry

Application:
Classification

Experiments

Adapted
gradient flow

Consider the cone $p + \mathfrak{C}_\theta[h] \subseteq \mathbb{R}^n$ centered around a unit vector h and based at p :



There are W and b such that, for $\sigma = \text{ReLU}$,

$$\tau_{W,b}(x) = \begin{cases} x, & x \in p + \mathfrak{C}_\theta[h], \\ p, & x \in p + \mathfrak{C}_\theta[-h]. \end{cases} \quad (3.13)$$

Truncation map and (polyhedral) cones for ReLU

Truncation
map

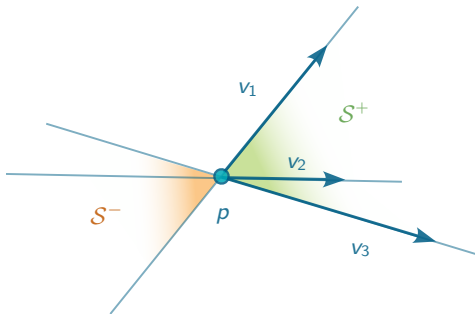
Theory

Application:
Symmetry

Application:
Classification

Experiments

Adapted
gradient flow



Given $W \in GL(n)$ and $b \in \mathbb{R}^n$, we can find p and $(v_i)_{i=1}^m \in \mathbb{R}^n$ such that

$$\tau_{W,b}(x) = \begin{cases} x, & x \in S^+, \\ p, & x \in S^-. \end{cases} \quad (3.14)$$

Multiclass classification

Truncation
map

Theory

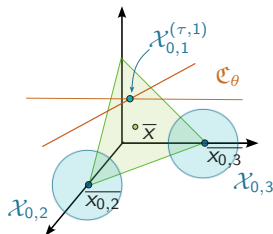
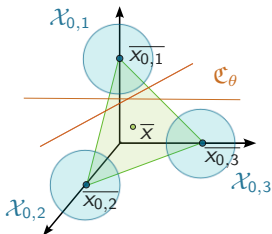
Application:
Geometry

Application:
Classification

Experiments

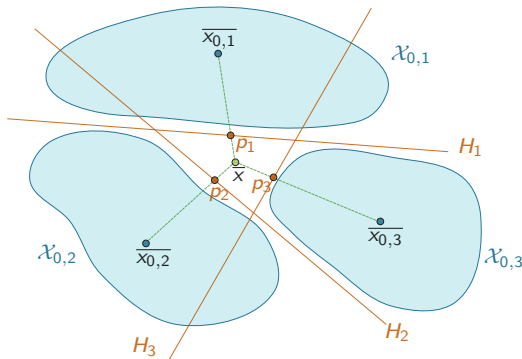
Adapted
gradient flow

Consider a set of training data $\mathcal{X}_0 = \bigcup_{j=1}^Q \mathcal{X}_{0,j} \subseteq \mathbb{R}^{d_0}$, with Q classes. If the data is sufficiently clustered, then we can construct a ReLU neural network with $Q + 1$ layers which interpolates the data.



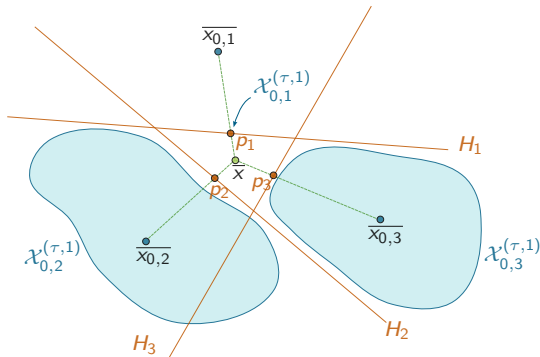
Definition (Chen–E. '24)

We say a dataset is **sequentially linearly separable** if there exists an ordering of the classes such that for each $j = 1, \dots, Q$, there exists a hyperplane H_j and a point $p_j \in H_j$ such that H_j separates $\mathcal{X}_{0,j}$ and $\bigcup_{k < j} \{p_k\} \cup \bigcup_{k > j} \mathcal{X}_{0,k}$.



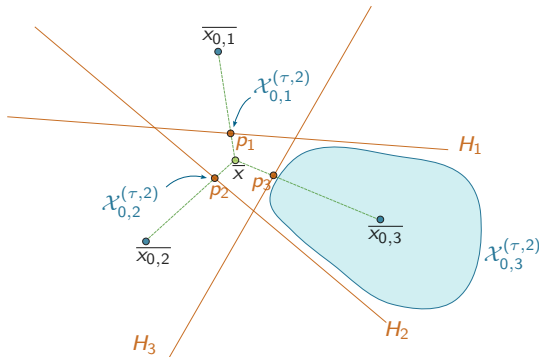
Definition (Chen–E. '24)

We say a dataset is **sequentially linearly separable** if there exists an ordering of the classes such that for each $j = 1, \dots, Q$, there exists a hyperplane H_j and a point $p_j \in H_j$ such that H_j separates $\mathcal{X}_{0,j}$ and $\bigcup_{k < j} \{p_k\} \cup \bigcup_{k > j} \mathcal{X}_{0,k}$.



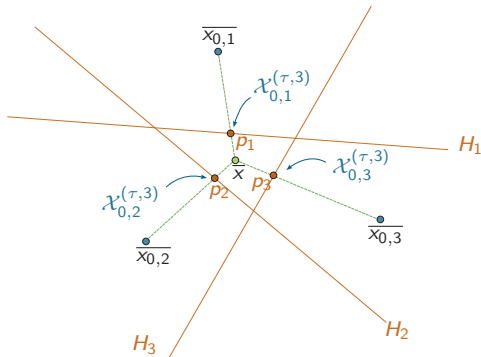
Definition (Chen–E. '24)

We say a dataset is **sequentially linearly separable** if there exists an ordering of the classes such that for each $j = 1, \dots, Q$, there exists a hyperplane H_j and a point $p_j \in H_j$ such that H_j separates $\mathcal{X}_{0,j}$ and $\bigcup_{k < j} \{p_k\} \cup \bigcup_{k > j} \mathcal{X}_{0,k}$.



Definition (Chen–E. '24)

We say a dataset is **sequentially linearly separable** if there exists an ordering of the classes such that for each $j = 1, \dots, Q$, there exists a hyperplane H_j and a point $p_j \in H_j$ such that H_j separates $\mathcal{X}_{0,j}$ and $\bigcup_{k < j} \{p_k\} \cup \bigcup_{k > j} \mathcal{X}_{0,k}$.



Binary classification: Non-separable data

Truncation
map

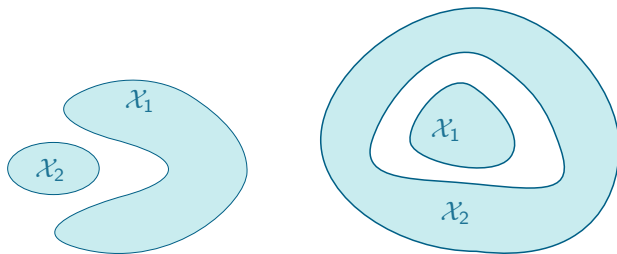
Theory

Application:
Geometry

Application:
Classification
Experiments

Adapted
gradient flow

We would like to obtain similar results for data that is not sequentially linearly separable. For simplicity, consider binary classification.



Binary classification: Non-separable data

Truncation
map

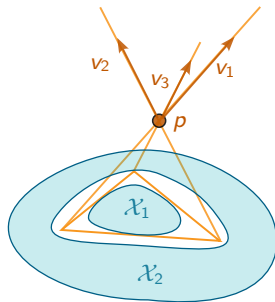
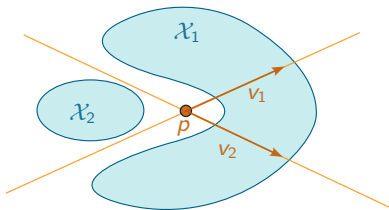
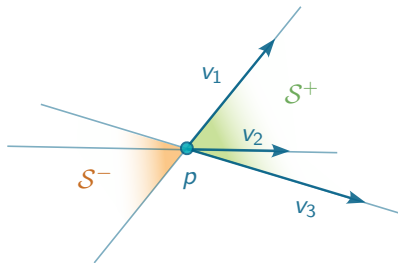
Theory

Application:
Geometry

Application:
Classification

Experiments

Adapted
gradient flow



Emerging Cone Structure

Truncation
map

Theory

Application

Symmetry

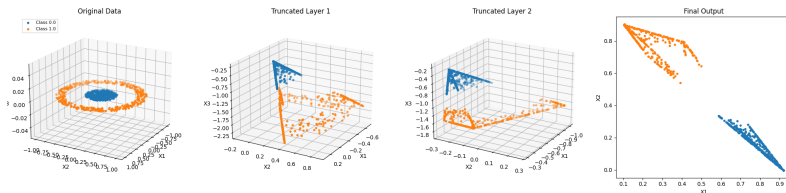
Application

Classification

Experiments

Adapted
gradient flow

In general, what do the cones of trained ReLU neural networks look like?



Videos.

This is work in progress with Thomas Chen and Mayank Konduri.

Gradient flow in output space

Truncation
map

Theory
Application
Geometry
Classification
Experiments

Adapted
gradient flow

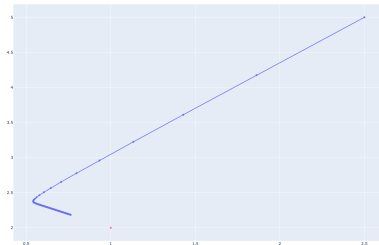
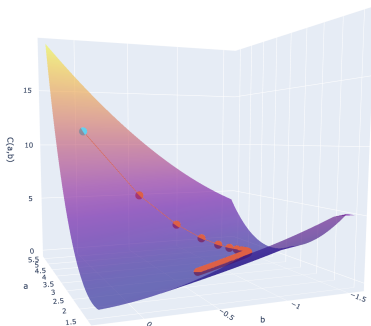
Gradient flow in parameter space

$$\frac{\partial \underline{\theta}}{\partial t} = -\nabla_{\underline{\theta}} \mathcal{C}$$

→

Induced flow in output space

$$\frac{\partial \underline{x}}{\partial t} = -DD^T \nabla_{\underline{x}} \mathcal{C}$$



Can the flow in output space be made (near) Euclidean? Yes, by adding a preconditioner.

Gradient flow in parameter space

$$\frac{\partial \underline{\theta}}{\partial t} = -\nabla_{\underline{\theta}} \mathcal{C}$$

$$\frac{\partial \underline{\theta}}{\partial t} = -(D^T D)^+ \nabla_{\underline{\theta}} \mathcal{C}$$

→

Induced flow in output space

$$\frac{\partial \underline{x}}{\partial t} = -DD^T \nabla_{\underline{x}} \mathcal{C}$$

$$\frac{\partial \underline{x}}{\partial t} = -\mathcal{P}_{\text{ran } D} \nabla_{\underline{x}} \mathcal{C},$$

Theorem (Chen–E. '24)

There is a homotopy between these gradient vector fields which preserves critical points.

This holds even if D is rank-deficient.

Can the flow in output space be made (near) Euclidean? Yes, by adding a preconditioner.

Gradient flow in parameter space

$$\frac{\partial \underline{\theta}}{\partial t} = -\nabla_{\underline{\theta}} \mathcal{C}$$

$$\frac{\partial \underline{\theta}}{\partial t} = -(D^T D)^+ \nabla_{\underline{\theta}} \mathcal{C}$$

→

Induced flow in output space

$$\frac{\partial \underline{x}}{\partial t} = -DD^T \nabla_{\underline{x}} \mathcal{C}$$

$$\frac{\partial \underline{x}}{\partial t} = -\mathcal{P}_{\text{ran } D} \nabla_{\underline{x}} \mathcal{C},$$

Theorem (Chen–E. '24)

There is a homotopy between these gradient vector fields which preserves critical points.

This holds even if D is rank-deficient.

Can the flow in output space be made (near) Euclidean? Yes, by adding a preconditioner.

Gradient flow in parameter space

$$\frac{\partial \underline{\theta}}{\partial t} = -\nabla_{\underline{\theta}} \mathcal{C}$$

$$\frac{\partial \underline{\theta}}{\partial t} = -(D^T D)^+ \nabla_{\underline{\theta}} \mathcal{C}$$

→

Induced flow in output space

$$\frac{\partial \underline{x}}{\partial t} = -DD^T \nabla_{\underline{x}} \mathcal{C}$$

$$\frac{\partial \underline{x}}{\partial t} = -\mathcal{P}_{\text{ran } D} \nabla_{\underline{x}} \mathcal{C},$$

Theorem (Chen–E. '24)

There is a homotopy between these gradient vector fields which preserves critical points.

This holds even if D is rank-deficient.

Can the flow in output space be made (near) Euclidean? Yes, by adding a preconditioner.

Gradient flow in parameter space

$$\frac{\partial \underline{\theta}}{\partial t} = -\nabla_{\underline{\theta}} \mathcal{C}$$

$$\frac{\partial \underline{\theta}}{\partial t} = -(D^T D)^+ \nabla_{\underline{\theta}} \mathcal{C}$$

→

Induced flow in output space

$$\frac{\partial \underline{x}}{\partial t} = -DD^T \nabla_{\underline{x}} \mathcal{C}$$

$$\frac{\partial \underline{x}}{\partial t} = -\mathcal{P}_{\text{ran } D} \nabla_{\underline{x}} \mathcal{C},$$

Theorem (Chen–E. '24)

There is a homotopy between these gradient vector fields which preserves critical points.

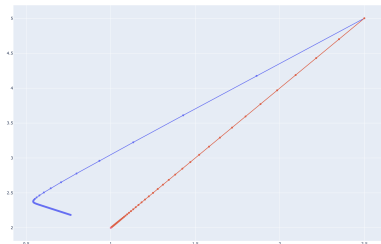
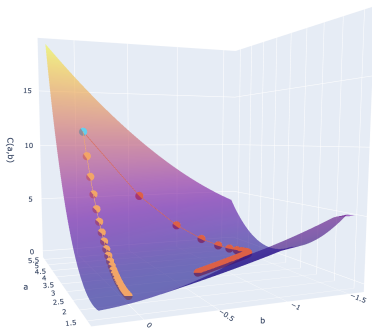
This holds even if D is rank-deficient.

Suppose D is surjective. Then

$$\frac{\partial \theta}{\partial t} = -(D^T D)^+ \nabla_{\underline{\theta}} \mathcal{C} \longrightarrow \frac{\partial \underline{x}}{\partial t} = -\nabla_{\underline{x}} \mathcal{C}, \quad (4.1)$$

and for commonly used loss functions the situation simplifies considerably.

Back to the simple example:

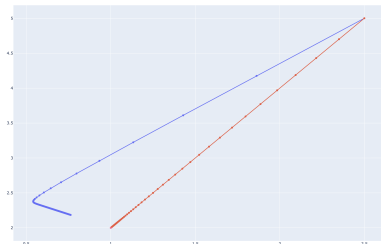
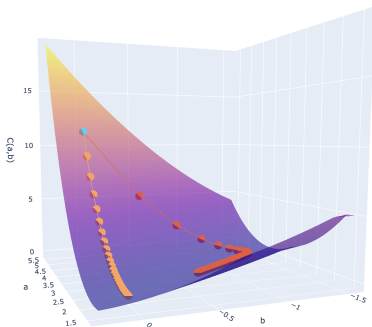


Suppose D is surjective. Then

$$\frac{\partial \theta}{\partial t} = -(D^T D)^+ \nabla_{\underline{\theta}} \mathcal{C} \longrightarrow \frac{\partial \underline{x}}{\partial t} = -\nabla_{\underline{x}} \mathcal{C}, \quad (4.1)$$

and for commonly used loss functions the situation simplifies considerably.

Back to the simple example:

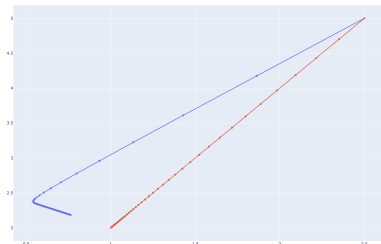
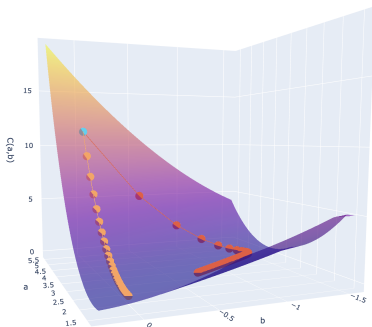


Suppose D is surjective. Then

$$\frac{\partial \theta}{\partial t} = -(D^T D)^+ \nabla_{\underline{\theta}} \mathcal{C} \longrightarrow \frac{\partial \underline{x}}{\partial t} = -\nabla_{\underline{x}} \mathcal{C}, \quad (4.1)$$

and for commonly used loss functions the situation simplifies considerably.

Back to the simple example:



What is known about the rank of D for neural networks?

- Highly depends on the architecture, and probably initialization.
- (Karhadkar et al. 24) show it's generically full-rank for 'mildly' overparametrized networks.
- (Grigsby et al. 22) show empirically that rank of D is low.

Potential applications of the homotopy result:

- Building block for rigorous results in optimization.
- New tool for experimentally exploring structure of loss landscape
→ Work in progress.

What is known about the rank of D for neural networks?

- Highly depends on the architecture, and probably initialization.
- (Karhadkar et al. 24) show it's generically full-rank for 'mildly' overparametrized networks.
- (Grigsby et al. 22) show empirically that rank of D is low.

Potential applications of the homotopy result:

- Building block for rigorous results in optimization.
- New tool for experimentally exploring structure of loss landscape
→ Work in progress.

What is known about the rank of D for neural networks?

- Highly depends on the architecture, and probably initialization.
- (Karhadkar et al. 24) show it's generically full-rank for 'mildly' overparametrized networks.
- (Grigsby et al. 22) show empirically that rank of D is low.

Potential applications of the homotopy result:

- Building block for rigorous results in optimization.
- New tool for experimentally exploring structure of loss landscape
→ Work in progress.

What is known about the rank of D for neural networks?

- Highly depends on the architecture, and probably initialization.
- (Karhadkar et al. 24) show it's generically full-rank for 'mildly' overparametrized networks.
- (Grigsby et al. 22) show empirically that rank of D is low.

Potential applications of the homotopy result:

- Building block for rigorous results in optimization.
- New tool for experimentally exploring structure of loss landscape
→ Work in progress.

What is known about the rank of D for neural networks?

- Highly depends on the architecture, and probably initialization.
- (Karhadkar et al. 24) show it's generically full-rank for 'mildly' overparametrized networks.
- (Grigsby et al. 22) show empirically that rank of D is low.

Potential applications of the homotopy result:

- Building block for rigorous results in optimization.
- New tool for experimentally exploring structure of loss landscape
→ Work in progress.

What is known about the rank of D for neural networks?

- Highly depends on the architecture, and probably initialization.
- (Karhadkar et al. 24) show it's generically full-rank for 'mildly' overparametrized networks.
- (Grigsby et al. 22) show empirically that rank of D is low.

Potential applications of the homotopy result:

- Building block for rigorous results in optimization.
- New tool for experimentally exploring structure of loss landscape
→ Work in progress.

Truncation map $\longrightarrow$ sequence of transformations on a fixed space:

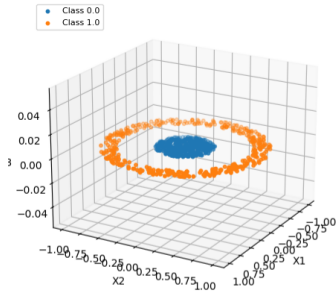
$$f_{\underline{\theta}} : \underbrace{\mathbb{R}^{d_0} \xrightarrow{\tau_1} \mathbb{R}^{d_0} \xrightarrow{\tau_2} \dots \xrightarrow{\tau_{L-1}} \mathbb{R}^{d_0}}_{\text{Truncation maps}} \xrightarrow{\tilde{f}_L} \mathbb{R}^{d_L}. \quad (5.1)$$

(Potential) applications: explicit constructions, neural collapse, parameter space symmetry, interpretability.

Gradient flow for parametrized functions can be continuously deformed to (constrained) Euclidean gradient flow in output space.

Thank you!

Original Data



Truncated Layer 1

